

# Dharma DROS: Establishing the Microkernel Traceability Operating System and Non-Repudiable AgentWeb Security Infrastructure for the Agent Era

Jui-Cheng(Jimmy) Chen  
Top-Celestial Company Ltd.  
Taipei, Taiwan R.O.C.  
jimmychen@dr-os.io

**Abstract**—Large Language Models (LLMs) inherently suffer from unpredictable hallucination problems, which is a fatal flaw in domains demanding extreme rigor. This research initially focused on developing an “AI Buddhist Reasoning System”; however, we discovered that models constrained by traditional prompts are highly prone to inventing fake scriptures or misinterpreting doctrines. To eradicate this issue, we developed the Dharma Reasoning OS: by introducing the Buddhist mechanism of “Panjiao (Doctrinal Classification)” as rigid constraint contracts, and utilizing C-FFI technology to intercept the LLM’s execution paths at the lowest level, we successfully achieved zero-hallucination Buddhist reasoning.

Building upon this success, we realized that this core architecture of “Contract Constraints + C-FFI Physical Blocking” is not only applicable to Buddhism but represents the ultimate solution for global AI Agent governance. Consequently, we evolved and generalized it into the DROS (Deterministic Runtime Operating System) architecture. DROS introduces the “Microkernel” philosophy of traditional operating systems, downgrading the LLM to a mere “Semantic Coprocessor,” while the memory-safe GuardVM retains absolute Ring 0 privileges. Through hardcoded Vajra Contracts and the proprietary T-Number Absolute Traceability coordinate system, DROS achieves 100% Auditability, laying an absolutely secure constitutional-level infrastructure for enterprise-grade Agent deployment.

**Index Terms**—Microkernel, AgentWeb, Zero-Trust, C-FFI, Traceability, Prompt Injection, Non-Repudiation

## I. Introduction: From “Doctrinal Classification” to “Deterministic Governance”

In the construction of autonomous AI Agents, the industry widely adopts monolithic frameworks like LangChain or AutoGen. These frameworks conflate semantic generation with system execution privileges, heavily relying on the LLM’s “self-awareness” to determine security boundaries. However, using a probabilistic model to constrain another probabilistic model inevitably faces the catastrophic risks of prompt injection and hallucination overstepping.

This pain point was infinitely magnified during our initial development of the “Digital Dharma Hall” AI reasoning system. The deduction of Buddhist doctrines demands absolute precision; any scriptures fabricated due to model hallucinations or arguments deviating from orthodox teachings are entirely unacceptable. Traditional

Retrieval-Augmented Generation (RAG) or Prompt constraints fundamentally fail to suppress the divergent instincts of large models.

To solve this conundrum, we traced back to the ancient Indian and Chinese Buddhist tradition of “Panjiao (Doctrinal Classification)” —establishing a rigorous system for evaluating doctrines and boundary standards. We developed the Dharma Reasoning OS, transforming the Panjiao mechanisms into enforceable “Constraint Contracts.” When the LLM performs Buddhist reasoning, the system establishes a choke point at the lowest level using C-FFI (C Foreign Function Interface). If the LLM’s generated inference attempts to cross the boundaries of the Panjiao contract, the C-FFI directly triggers a “Physical Melt” at the hardware level, rejecting the output. This mechanism successfully birthed the world’s first hallucination-free AI Buddhist reasoning system.

From Dharma to Deterministic: After witnessing the absolute power of this defense mechanism, we further abstracted its technological core. We found that the “Buddhist Panjiao Contract” is fundamentally identical to “Enterprise Cybersecurity Compliance Policies.” Therefore, we generalized and upgraded the Dharma Reasoning OS into DROS (Deterministic Runtime Operating System).

It abandons blind trust in neural networks and returns to the most classic security design in computer science—the microkernel architecture—providing an ultimate, cross-model, and cross-platform governance framework for Agent applications in all domains.

## II. Core Philosophy: The Microkernel Approach

In the history of operating system development, the microkernel ( $\mu$ -kernel) architecture is renowned for its extreme security isolation and high stability. DROS perfectly transplants this philosophy into the realm of AI governance:

- 1) **Minimal Core:** The core engine of DROS, GuardVM, is strictly limited to a few hundred lines of code and is written in memory-safe languages (e.g., Rust, Go, C++). This tremendously minimizes the potential Attack Surface.

- 2) Separation of Concerns: “Thinking” occurs outside the kernel (handled by the LLM), while “Rule Enforcement” occurs strictly inside the kernel (handled by the CPU).
- 3) Fail-Closed Security: By default, all unverified LLM actions are dropped. Only when an action is explicitly permitted by the Vajra Contract and accompanied by a valid T-Number credential will it be passed to the execution layer.

### III. Architecture Deep Dive

The DROS system is constructed from four critical layers, forming an impenetrable defense net:

#### A. Semantic Coprocessor

In the DROS worldview, the status of the LLM (e.g., GPT-4, Claude 3) is “downgraded.” It is no longer the omnipotent “Decision Maker” but merely a “Semantic Coprocessor.” It is fed context and requested to generate potential action drafts, but it possesses zero direct capability to access APIs, databases, or client outputs.

#### B. GuardVM (Supervisor Ring 0)

GuardVM is the privileged engine running on the host machine, responsible for intercepting the streaming byte output of the LLM.

- It maintains the State Machine of the current conversation.
- It parses incoming LLM tokens and continuously compares them against the loaded Vajra Contract.
- If the token stream violates a Regular Expression (Regex), semantic boundary, or data-exfiltration rule, GuardVM instantly terminates the TCP/WebSocket connection with the LLM.

#### C. Vajra Contract (Hardcoded Constraints)

Unlike System Prompts, which an LLM can “forget” or bypass at any time, the Vajra Contract is a human-readable Markdown or YAML file written by domain experts. It acts like hardware ROM, defining the absolute boundaries of the system. These contracts do not rely on the LLM for parsing; instead, they are evaluated directly by the host CPU via deterministic C-FFI / Rust logic.

#### D. VajraClaw Adapter

To support multiple languages and commercial deployment, DROS utilizes C-FFI (C Foreign Function Interfaces) technology. Whether it is a data science team’s Python script, a Web backend’s Node.js, or an enterprise-grade Java system, all can enjoy identical, physically-enforced protection with zero semantic drift via compiled shared libraries (e.g., vajra\_claw.so).

### IV. Absolute Traceability & Non-Repudiable Cryptographic Signatures

The core requirement of the DROS constitution is Absolute Traceability and Non-Repudiation. When the LLM makes any factual statement or tool execution, it must provide a precise coordinate pointing to the authorizing clause in the Vajra Contract, which is then cryptographically stamped by the system.

Operational Mechanism:

- 1) Human expert source documents (e.g., enterprise security protocols) are parsed by DROS, and each paragraph/rule is assigned a unique coordinate known as the T-Number (e.g., [T1-045]).
- 2) The LLM is subjected to strict low-level instruction constraints: “For every declaration or action, the authorizing T-Number must be appended.”
- 3) GuardVM intercepts the output. If it detects an action missing a T-Number, or if the T-Number logically disallows the action (evaluated deterministically by the CPU), the output is immediately Melted.
- 4) DROS-by-execution PKI: If the action is permitted, the system utilizes asymmetric cryptography to issue a tamper-proof digital certificate for that specific micro-execution (Per-Execution).

This mechanism perfectly satisfies stringent legal and financial compliance audits like HIPAA and SOC2, because every micro-action of the AI is not only physically anchored to human-signed documents but also leaves an absolutely tamper-proof and undeniable cryptographic ironclad evidence.

### V. Physical Melt vs. Prompt Engineering

DROS’s Physical Melt mechanism has profound fundamental differences compared to traditional defenses, as shown in Table I:

TABLE I  
Comparison between traditional probabilistic defenses and DROS deterministic Physical Melt

| Feature               | Prompt Engineering / RAG Soft Boundaries        | DROS Physical Melt                           |
|-----------------------|-------------------------------------------------|----------------------------------------------|
| Enforcement Layer     | LLM Neural Network Weights (Probabilistic)      | Host CPU Memory (Deterministic)              |
| Response to Injection | May apologize and comply if deceived            | Instantly kills the TCP connection (abort()) |
| Token Cost for Safety | High (Requires secondary LLM-as-a-judge passes) | Zero (Evaluated via Regex/AST on CPU)        |
| Auditability          | Black Box                                       | 100% Transparent (T-Number Coordinates)      |

## VI. Deployment Topologies & Global Coverage

This infrastructure is designed to seamlessly adapt to multi-layered deployment topologies, allowing the AgentWeb security net to cover every corner from the cloud to the terminal:

- 1) Cloud-Native Edge: Suitable for ultra-low latency Web applications, deployed on Kubernetes or Cloudflare Workers.
- 2) Enterprise Internal API: Deployed within an enterprise VPC for internal AI assistants used by HR, Legal, or Finance.
- 3) Air-Gapped Sovereign: For military defense, Fortune 500 companies, or highly regulated sovereign networks, providing 100% offline operation, Zero Telemetry, and hardware UUID binding.
- 4) Mobile & Edge SDK: Packaging the core C-FFI interception engine into an extremely lightweight VajraClaw mobile SDK, allowing the defense net to be deployed directly on users' iOS or Android devices. This ultimate protection, extending from cloud clusters straight into people's pockets, ensures AgentWeb is governed by the same deterministic standard at every level.

## VII. Conclusion & Future Work

The Dharma DROS (first installment) redefines the "Operating System Constitution" of the Agent Era. By discarding reliance on probabilistic models and introducing microkernel philosophy, T-Number coordinate systems, and physical melt mechanisms, DROS successfully tames the LLM from an "uncontrollable black-box decision maker" into a "safely controlled semantic coprocessor."

However, single-node security governance is merely the starting point. As AI enters the global collaborative network, the next challenge we face is how to establish mutual trust between Agents across different trust domains. Therefore, the microkernel and T-Number traceability foundations laid by this research naturally lead to the next core milestone of the DROS system: the AgentWeb Trust Network and Non-Repudiable Cryptographic Signatures (DROS-by-Execution PKI) infrastructure.

By imprinting cryptographically valid certificate stamps on every minute execution action (Per-Execution) of large models, we will realize an undeniably traceable secure collaborative network in the uncertain Agent Era—"leave a trace wherever you go." This ultimate infrastructure, combining "Dharma constraints" and "Zero-Trust cryptography," will lead human society safely into the next golden decade of AI.

## References

- [1] J. Liedtke, "On Micro-Kernel Construction," ACM SIGOPS Operating Systems Review, 1995.
- [2] S. Bubeck et al., "Sparks of Artificial General Intelligence: Early experiments with GPT-4," arXiv, 2023.
- [3] K. Greshake et al., "More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models," 2023.

- [4] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020.

## Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work and the development of the prototype system, the author utilized large language models (including but not limited to the Google Gemini series) to assist with academic literature translation, code framework generation, and English formatting/proofreading. The author takes full responsibility for all logical architectures, core philosophies (including microkernel architecture and T-Number absolute traceability), and final content of this paper, and has conducted strict review and verification of all AI-generated content.